

FarmaMuni (KoreAPP) — Documentación de codificación

Capítulo 5. Fragmentos de código relevantes del sistema de gestión farmacéutica municipal. Stack: Next.js App Router, TypeScript, Supabase, TanStack Query, Zod, Tailwind SIGET.

5.1.1 Registro de módulos de negocio (APP_MODULES)

```
export const APP_MODULES: AppModuleConfig[] = [
  {
    id: "ventas",
    title: "Ventas",
    subtitle: "",
    desc: "Punto de venta, control de caja diaria, reportes de ingresos y facturación.",
    href: `${APP_BASE_PATH}/ventas`,
    allowedRoles: ["super", "admin", "ventas", "user"],
    bento: "col-span-1 row-span-2 md:col-span-2 md:row-span-2",
    size: "hero",
    tag: "Facturación · Caja",
    cardBg: "bg-white hover:bg-zinc-50 dark:bg-zinc-900 dark: hover:bg-zinc-800/90",
    cardBorder: "border-zinc-200 dark:border-zinc-800",
  },
  {
    id: "clientes",
    title: "Clientes",
    subtitle: "",
    desc: "Directorio de clientes, historial de compras, saldos y fichas de contacto.",
    href: `${APP_BASE_PATH}/clientes`,
    allowedRoles: ["super", "admin", "clientes", "ventas"],
    bento: "col-span-1 row-span-1 md:col-start-3 md:row-start-1",
    size: "compact",
    tag: "Directorio · Historial",
    cardBg: "bg-white hover:bg-zinc-50 dark:bg-zinc-900 dark: hover:bg-zinc-800/90",
    cardBorder: "border-zinc-200 dark:border-zinc-800",
  },
  {

```

Figura 1. Configuración declarativa de módulos FarmaMuni

El arreglo APP_MODULES centraliza rutas, roles permitidos y metadatos de tarjetas del panel principal.

Cada objeto define href bajo /farmamuni, etiquetas visuales y restricciones allowedRoles para el menú lateral.

Esta capa desacopla la navegación de los componentes de página y facilita agregar módulos sin duplicar lógica.

5.1.2 Server Action — consulta de inventario

```
export async function obtenerProductos() {
  try {
    const supabase = await createClient();
    const { data: { user } } = await supabase.auth.getUser();
    if (!user) return { code: "UNAUTHORIZED" as const };

    const { data, error } = await supabase
      .from("inv_productos")
      .select("*, inv_proveedores(nombre), inv_compras_detalle(inv_compras(inv_proveedores(nombre)))")
      .order("nombre", { ascending: true });

    if (error) return { code: "INTERNAL" as const };
    return { success: true as const, data: data ?? [] };
  } catch {
    return { code: "INTERNAL" as const };
  }
}

export async function obtenerProducto(id: string) {
  try {
    const supabase = await createClient();
    const { data: { user } } = await supabase.auth.getUser();
    if (!user) return { code: "UNAUTHORIZED" as const };

    const { data, error } = await supabase
      .from("inv_productos")
      .select("*, inv_proveedores(nombre)")
      .eq("id", id)

```

Figura 2. Obtención segura de productos con Supabase

Las acciones de servidor validan sesión con `supabase.auth.getUser()` antes de leer datos.

`obtenerProductos` realiza un select con relaciones a proveedores y detalles de compra.

Los errores se normalizan a códigos controlados (UNAUTHORIZED, INTERNAL) para el cliente.

5.1.3 Esquema Zod de producto

```
import { z } from "zod";

export const productSchema = z.object({
  nombre: z.string().min(1, "El nombre es requerido"),
  descripcion: z.string().optional(),
  precio_base: z.number().nonnegative("El precio debe ser un número positivo"),
  stock_minimo: z.number().nonnegative("El stock mínimo debe ser un número no negativo"),
  activo: z.boolean().default(true),
  imagen_url: z.string().nullable().optional(),
  proveedor_id: z.string().nullable().optional(),
});

export type ProductFormValues = z.infer<typeof productSchema>;

export const bajaVencidoSchema = z.object({
  lote_id: z.string().uuid(),
  notas: z.string().trim().max(300).optional(),
});

export type BajaVencidoInput = z.infer<typeof bajaVencidoSchema>;

export const crearLoteManualSchema = z.object({
  producto_id: z.string().uuid(),
  codigo_barras: z.string().trim().min(1, "El código de barras es requerido"),
  numero_lote: z.string().trim().min(1, "El número de lote es requerido"),
  cantidad: z.number().positive("La cantidad debe ser mayor a 0"),
  precio_costo: z.number().nonnegative("El costo debe ser un número válido"),
  fecha_vencimiento: z.string().trim().min(1, "La fecha de vencimiento es requerida"),
});
```

Figura 3. Validación de formularios de inventario

productSchema concentra tipos inferidos con z.infer; no se usan interfaces sueltas en el módulo.

Los campos numéricos y fechas se validan antes de mutar en Supabase.

safeParse en actions evita exponer errores crudos de base de datos al usuario.

5.2.1 Carga de productos y clientes para ventas

```
export async function obtenerProductosYClientes() {
  try {
    const supabase = await createClient();

    // Obtener productos activos
    const { data: productos, error: prodError } = await supabase
      .from("inv_productos")
      .select("*")
      .eq("activo", true)
      .order("nombre", { ascending: true });

    if (prodError) throw new Error(prodError.message);

    // Obtener todos los clientes
    const { data: clientes, error: cliError } = await supabase
      .from("ven_clientes")
      .select("*")
      .order("nombre", { ascending: true });

    if (cliError) throw new Error(cliError.message);

    return {
      productos: productos || [],
      clientes: clientes || [],
    };
  } catch (error: any) {
    console.error("Error en obtenerProductosYClientes:", error);
    throw new Error("No se pudieron cargar los productos o clientes.");
  }
}
```

Figura 4. Agregación de catálogo en punto de venta

obtenerProductosYClientes une inv_productos activos y ven_clientes ordenados.

El POS consume este endpoint vía TanStack Query en el cliente.

Centralizar la carga reduce round-trips y mantiene consistencia de precios y stock.

5.2.2 Contexto global de ventas

```
const [clienteBusqueda, setClienteBusqueda] = useState("Consumidor Final");
const [clienteSeleccionado, setClienteSeleccionado] = useState<Cliente | null>(null);
const [mostrarSugerenciasCli, setMostrarSugerenciasCli] = useState(false);
const [isCrearClienteOpen, setIsCrearClienteOpen] = useState(false);

const [tipoVenta, setTipoVenta] = useState<"Contado" | "Crédito">("Contado");
const [mostrarMetodoPagoDropdown, setMostrarMetodoPagoDropdown] = useState(false);
const [observaciones, setObservaciones] = useState("");
const [isProcesandoVenta, setIsProcesandoVenta] = useState(false);
const [showUbicacionModal, setShowUbicacionModal] = useState(false);

const [editingCartItemIndex, setEditingCartItemIndex] = useState<number | null>(null);
const [editingPrice, setEditingPrice] = useState<string>("");
const [editingQty, setEditingQty] = useState<string>("");
const [animateCart, setAnimateCart] = useState(false);

const [ticketParaImprimir, setTicketParaImprimir] = useState<any>(null);
const [reciboCaptura, setReciboCaptura] = useState<any>(null);
const [reciboModalData, setReciboModalData] = useState<any>(null);

const [solicitudRebajaId, setSolicitudRebajaId] = useState<string | null>(null);
const [esperandoAutorizacionRebaja, setEsperandoAutorizacionRebaja] = useState(false);
const [rebajaAutorizada, setRebajaAutorizada] = useState(false);
const [showModalAutorizacionRebaja, setShowModalAutorizacionRebaja] = useState(false);
const [isValidandoAutorizacionRebaja, setIsValidandoAutorizacionRebaja] = useState(false);
const carritoSnapshotRef = useRef<string>("");
```

Figura 5. Estado del carrito y tipo de venta

ContextoVentas expone carrito, cliente seleccionado y tipo Contado/Crédito a toda la UI de ventas.

El proveedor React evita prop drilling entre SeccionProductos, sidebar y modales.

Las reglas de negocio (crédito requiere cliente) se aplican en el contexto antes de persistir.

5.2.3 Filtros de historial de ventas

```
export function fechaVentaCalendarioGt(createdAt: string | null | undefined): string {
  if (!createdAt) return "";
  const date = new Date(createdAt);
  if (Number.isNaN(date.getTime())) return "";
  return fechaCalendarioGt(date);
}

function claveTipoVentaHistorial(tipoVenta: string | null | undefined): string {
  return (tipoVenta ?? "")
    .trim()
    .toLowerCase()
    .normalize("NFD")
    .replace(/[\u0300-\u036f]/g, "");
}

export function ventaEsCreditoHistorial(tipoVenta: string | null | undefined): boolean {
  return claveTipoVentaHistorial(tipoVenta) === "credito";
}

export function ventaEsContadoHistorial(tipoVenta: string | null | undefined): boolean {
  if (ventaEsCreditoHistorial(tipoVenta)) return false;
  const key = claveTipoVentaHistorial(tipoVenta) || "contado";
  return ["efectivo", "contado", "transferencia", "tarjeta"].includes(key);
}

export function etiquetaTipoVentaHistorial(tipoVenta: string | null | undefined): string {
  if (ventaEsCreditoHistorial(tipoVenta)) return "Crédito";
  if (ventaEsContadoHistorial(tipoVenta)) {
```

Figura 6. Normalización de tipo de pago en historial

fechaVentaCalendarioGt alinea fechas a zona America/Guatemala para filtros por día.

ventaCoincideFiltroPagoHistorial clasifica contado/crédito ignorando mayúsculas y tildes.

Esta capa pura facilita pruebas y evita duplicar condiciones en el componente de tabla.

5.3.1 Hooks de clientes (TanStack Query)

```
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
import { toast } from "react-toastify";
import { useDemoMode } from "@components/(base)/providers/DemoModeProvider";
import {
  DEMO_CLIENTES,
  DEMO_VENTAS_HISTORIAL,
} from "@lib/demo/fixtures";
import {
  assertWritableDemo,
  demoQueryKey,
  resolveDemoData,
} from "@lib/demo/helpers";
import { obtenerClientes, obtenerVentasCliente, crearCliente, editarCliente } from "./actions";
import { ClienteInput } from "./zod";

export function useClientes() {
  const { isDemoMode } = useDemoMode();
  return useQuery({
    queryKey: demoQueryKey(["clientes"], isDemoMode),
    queryFn: () => {
      resolveDemoData(
        isDemoMode,
        async () => {
          const res = await obtenerClientes();
          if (!res.success) throw new Error(res.code);
          return res.data;
        },
        DEMO_CLIENTES,
      );
    },
  });
}
```

Figura 7. Patrón de datos asíncronos en clientes

useClientes encapsula fetch, cache y revalidación con queryKey estable.

El modo demo intercambia fixtures locales sin cambiar la UI.

Los componentes VerClientes solo consumen hooks; no llaman Supabase directamente.

5.3.2 Panel historial de compras del cliente

```
return (
  <motion.div
    initial={{ opacity: 0, x: 32 }}
    animate={{ opacity: 1, x: 0 }}
    exit={{ opacity: 0, x: 32 }}
    className={cn(
      "flex h-full min-h-0 w-full flex-col overflow-hidden rounded-[2rem] bg-white shadow-2xl
dark:bg-zinc-900",
      className,
    )}
  >
    <div className="p-5 border-b border-zinc-100 dark:border-zinc-800 flex flex-col md:flex-row
justify-between items-start md:items-center gap-4 bg-zinc-50 dark:bg-zinc-800/50 shrink-0">
      <div>
        <h3 className="font-black text-zinc-800 dark:text-zinc-100 uppercase tracking-wider text-sm flex
items-center gap-2">
          <Clock className="size-4" /> Historial de Compras
        </h3>
        <p className="text-xs text-zinc-500 mt-1">Cliente: {cliente.nombre}</p>
      </div>

      <div className="flex-1 flex justify-center w-full md:w-auto">
        <div className="flex flex-col sm:flex-row items-center justify-center gap-3 w-full md:w-fit mx-auto
sm:mx-0">
          <div className="flex items-center justify-center gap-1 bg-slate-50 dark:bg-zinc-900/50 p-1
rounded-xl border border-slate-100 dark:border-zinc-800 w-full sm:w-auto">
            {[
              { id: "dia", label: "Mes/Año" },
              { id: "semana", label: "Mes" },
              { id: "rango", label: "Rango" },
            ].map((opt) => (
              <button
                key={opt.id}
```

Figura 8. Layout de overlay 70/30 en detalle de cliente

HistorialComprasPanel combina filtros de fecha, gráfica Recharts y listado filtrado.

El motion.div aplica animación de entrada coherente con el design system SIGET.

La estructura se reutilizó como referencia para el detalle de proveedores.

5.4.1 Creación de compra a proveedor

```
    } catch {
      return { code: "INTERNAL" as const };
    }
  }

export async function crearCompra(input: CompraInput) {
  try {
    const supabase = await createClient();
    const { data: { user } } = await supabase.auth.getUser();
    if (!user) return { code: "UNAUTHORIZED" as const };

    const parsed = CompraSchema.safeParse(input);
    if (!parsed.success) return { code: "VALIDATION" as const };

    const { proveedor_id, total, estado_pago, observaciones, items } = parsed.data;
    const pagado = esEstadoPagado(estado_pago);

    const { data: compra, error: compraError } = await supabase
      .from("inv_compras")
      .insert({
        proveedor_id,
        total,
        estado_pago: pagado ? "Pagado" : estado_pago,
        fecha_pago: pagado ? new Date().toISOString() : null,
        observaciones: observaciones?.trim() || null,
      })
      .select("id")
      .single();
  }
```

Figura 9. Transacción de compra e inventario

crearCompra valida payload con Zod, inserta inv_compras y detalles, y actualiza stock por lote.

revalidatePath mantiene sincronizadas las rutas de proveedores e inventario.

Los pagos parciales generan fin_transacciones con categoría pago_proveedor.

5.4.2 Shell de pestañas de compras

```
const COMPRAS_TABS = [
  { id: "ingresar_compra" as const, label: "Registrar" },
  { id: "proveedores" as const, label: "Proveedores" },
  { id: "historial" as const, label: "Historial" },
  { id: "cuentas_por_pagar" as const, label: "Por pagar" },
];

type ComprasTabId = (typeof COMPRAS_TABS)[number]["id"];

function comprasTabPillClass(active: boolean) {
  return cn(
    "shrink-0 cursor-pointer whitespace-nowrap rounded-lg px-3 py-2.5 text-[10px] font-bold uppercase tracking-wider transition-all sm:px-4 sm:text-xs",
    active
      ? "bg-white text-[#8DA78E] shadow-sm dark:bg-[#525D53] dark:text-white"
      : "text-zinc-500 hover:text-zinc-700 dark:hover:text-zinc-300",
  );
}

function VerProveedoresInner() {
  const [activeTab, setActiveTab] = useState<ComprasTabId>("ingresar_compra");
```

Figura 10. Navegación interna del módulo compras

COMPRAS_TABS define registrar, proveedores, historial y cuentas por pagar.

El estado activeTab vive en el contenedor; cada pestaña monta su sección sin rutas hijas.

Los estilos pill siguen el mismo patrón visual que créditos y finanzas.

5.4.3 Detalle de proveedor con historial integrado

```
        compras={compras}
        onClose={onClose}
      />
    </div>
    <motion.div
      initial={{ x: "100%" }}
      animate={{ x: 0 }}
      exit={{ x: "100%" }}
      transition={{ type: "spring", damping: 25, stiffness: 200 }}
      className="relative flex h-full w-full min-w-0 flex-col overflow-hidden rounded-[2rem] bg-white
shadow-2xl dark:bg-zinc-900 md:flex-[3]"
    >
      <ProveedorDetallePanel
        formData={formData}
        proveedor={proveedor}
        compras={compras}
        isEditing={isEditing}
        setFormData={setFormData}
        areaCode={areaCode}
        setAreaCode={setAreaCode}
        telefonoVal={telefonoVal}
        setTelefonoVal={setTelefonoVal}
        isSaving={isSaving}
        confirmEliminar={confirmEliminar}
        setConfirmEliminar={setConfirmEliminar}
        isDeleting={isDeleting}
        onClose={onClose}
        onOpenHistorial={() => setHistorialMovilAbierto(true)}
        onEdit={() => setIsEditing(true)}
      />
    </motion.div>
  </div>
</div>
```

Figura 11. Overlay fijo y panel dividido

VerProveedor abandona ModalShell estrecho por layout full-screen alineado a clientes.

useComprasProveedor carga historial vía server action autenticada.

En móvil el historial se abre en capa superior con AnimatePresence.

5.5.1 Resumen financiero en servidor

```
"use server";

import { createClient } from "@utils/supabase/server";
import { revalidatePath } from "next/cache";
import {
  registrarMovimientoSchema,
  type TransaccionFinanciera,
  type ResumenFinanciero,
  type FiltroTipo,
  type CuentaPorCobrar,
  type CuentaPorPagar,
} from "../zod";

const FINANZAS_PATH = "/farmamuni/finanzas";
const DEFAULT_PAGE_SIZE = 50;
const MAX_PAGE_SIZE = 200;

function toErrorMessage(error: unknown, fallback: string): string {
  if (error instanceof Error) return error.message;
  return fallback;
}

function sanitizeSearchTerm(term: string): string {
  return term.trim().replace(/[%_,]/g, (match) => `\\${match}`);
}

function rpcFinanzasNoDisponible(message: string): boolean {
  return (
```

Figura 12. Agregación de ingresos y egresos

Las finanzas calculan totales desde `fin_transacciones` con filtros de fecha en SQL o post-proceso.

Se distinguen movimientos ligados a ventas, compras y gastos fijos.

El cliente solo renderiza KPIs; la lógica sensible permanece en el servidor.

5.5.2 UI de control financiero

```
return (
  <div className={moduleListPageShellClass}>
    <div className="flex flex-col gap-3">
      <div className="flex flex-col md:flex-row justify-between items-start md:items-end gap-4 px-1">
        <div>
          <h1 className="text-2xl md:text-3xl font-black tracking-tight text-zinc-900 dark:text-white flex items-center gap-3">
            <div className="p-2.5 bg-[#8DA78E]/10 dark:bg-[#8DA78E]/20 text-[#8DA78E] rounded-xl">
              <Wallet className="size-8" />
            </div>
            Control Financiero
          </h1>
          <p className="text-sm text-muted-foreground mt-1 font-medium ml-1">
            Ingresos, egresos, balance y cuentas pendientes
          </p>
        </div>

        <div className="flex items-center gap-2 w-full md:w-auto">
          <SigetActionButton
            label="Ingreso"
            accentColor={sigetAccent.guardar}
            morphFrom={PlusNode}
            morphTo={TrendingUpNode}
            onClick={() => handleOpenNuevo("ingreso")}
            className="w-auto shrink-0 flex-1 md:flex-none"
          />
          <SigetActionButton
            label="Egreso"
            accentColor={sigetAccent.quitar}
          />
        </div>
      </div>
    </div>
  </div>
)
```

Figura 13. Encabezado y tarjetas de balance

moduleListPageShellClass reduce padding superior para alinear con otros módulos.

CustomDatePicker reemplaza entrada manual en filtros Día y Rango.

SigetActionButton dispara modales de ingreso y egreso con el tema SIGET.

5.6.1 Créditos — consulta de ventas a crédito

```
"use server";

import { createClient } from "@utils/supabase/server";
import type { CreditoResumen, VentaCreditoDetalle } from "../zod";

export async function obtenerResumenCreditos(): Promise<CreditoResumen[]> {
  try {
    const supabase = await createClient();

    // 1. Get all clients
    const { data: clientesData, error: cliError } = await supabase
      .from("ven_clientes")
      .select("*")
      .order("nombre", { ascending: true });

    if (cliError) throw new Error(cliError.message);

    // 2. Get all credit sales (without nested join to avoid timeout)
    const { data: ventasData, error: ventasError } = await supabase
      .from("ventas")
      .select("id, cliente_id, total, tipo_venta, created_at")
      .eq("tipo_venta", "Crédito")
      .order("created_at", { ascending: false })
      .limit(2000);

    if (ventasError) throw new Error(ventasError.message);
  }
}
```

Figura 14. Listado de cartera por cobrar

Las acciones filtran ven_ventas con tipo_venta Crédito y calculan saldos con abonos.

Los resultados alimentan tablas paginadas en VerCreditos.

RLS en Supabase limita filas según el tenant de la farmacia.

5.7.1 Layout compartido de módulos

```
export const moduleContentMaxWidth = "max-w-screen-2xl";

export const modulePageShellClass = [
  "w-full",
  moduleContentMaxWidth,
  "mx-auto flex flex-col gap-6",
  "px-3 sm:px-4 md:px-5",
  "pt-32 pb-8 md:pt-28",
  "relative mt-4 md:mt-8 min-h-screen",
].join(" ");

export const modulePageShellFixedClass = [
  "w-full",
  moduleContentMaxWidth,
  "mx-auto flex flex-col gap-6",
  "px-3 sm:px-4 md:px-5",
  "pt-32 pb-8 md:pt-28",
  "relative mt-4 md:mt-8 flex-1 min-h-0 h-screen",
].join(" ");

export const dashboardOuterClass =
  "relative z-10 w-full flex-1 px-3 sm:px-4 md:px-5 lg:px-6 pt-20 md:pt-24 pb-16 md:pb-20";

export const dashboardInnerClass = ["w-full", moduleContentMaxWidth, "mx-auto"].join(" ");

export const adminPageShellClass = [
  "space-y-8 w-full",
  moduleContentMaxWidth,
]
```

Figura 15. Clases Tailwind reutilizables

modulePageShellClass y moduleListPageShellClass estandarizan márgenes y max-width.

moduleControlsShellClass envuelve barras de filtros con borde y sombra SIGET.

Centralizar clases evita drift visual entre inventario, ventas y finanzas.

5.7.2 Modal SIGET (ModalShell)

```
"use client";

import { useEffect, useState, type ReactNode } from "react";
import { createPortal } from "react-dom";
import { AnimatePresence, motion } from "framer-motion";
import { Ban as BanNode, Check as CheckNode, Save, Trash, Trash2, X } from "lucide";
import {
  AlertTriangle,
  Ban as BanIcon,
  Package,
  Trash2 as Trash2Icon,
  X as XIcon,
} from "lucide-react";
import { SigetActionButton, sigetAccent } from "@components/ui/siget-action-button";
import {
  formatFechaManualGt,
  parseFechaManualGt,
} from "@lib/fechas-gt";
import { cn } from "@lib/utils";

const MODAL_SHELL_EASE = [0.22, 1, 0.36, 1] as const;

const MODAL_SHELL_TRANSITION = {
  duration: 0.24,
  ease: MODAL_SHELL_EASE,
} as const;

function lockBodyScroll() {
```

Figura 16. Componente base de formularios flotantes

ModalShell unifica header, footer y campos ModalInput/ModalFechaInput.

modalAccentClass aplica color institucional Trifinio en títulos y labels.

Prohibido usar Dialog shadcn en formularios nuevos según reglas del proyecto.

5.7.3 Fechas en zona horaria Guatemala

```
export const TIMEZONE_GT = "America/Guatemala" as const;
export const LOCALE_GT = "es-GT" as const;

const fechaCalendarioRegex = /^(\d{4})-(\d{2})-(\d{2})$/;
const mesCalendarioRegex = /^(\d{4})-(\d{2})$/;

export function fechaCalendarioGt(fecha = new Date()): string {
  return new Intl.DateTimeFormat("en-CA", {
    timeZone: TIMEZONE_GT,
    year: "numeric",
    month: "2-digit",
    day: "2-digit",
  }).format(fecha);
}

export function mesCalendarioGt(fecha = new Date()): string {
  return fechaCalendarioGt(fecha).slice(0, 7);
}

export function normalizarFechaCalendario(
  value: string | null | undefined,
): string {
  if (!value) return "";
  const solo = value.split("T")[0];
  return fechaCalendarioRegex.test(solo) ? solo : "";
}

export function normalizarMesCalendario(
```

Figura 17. Utilidad `fechaCalendarioGt`

`TIMEZONE_GT` fija `America/Guatemala` para todos los filtros por día.

`fechaCalendarioGt` formatea con `en-CA` (YYYY-MM-DD) independiente del locale del navegador.

`normalizarFechaCalendario` sanea strings ISO antes de comparar en filtros.

5.8.1 Enrutamiento App Router

```
import { Suspense } from "react";
import { VerVentas } from "@components/(base)/ventas/VerVentas";

export default function VentasPage() {
  return (
    <Suspense fallback={null}>
      <VerVentas />
    </Suspense>
  );
}
```

Figura 18. Página delgada con Suspense

Las rutas bajo `src/app/farmamuni` importan un único componente de dominio.

`page.tsx` no contiene lógica de negocio; solo composición y loading boundaries.

Este patrón se repite en clientes, inventario, proveedores y finanzas.